

# Kapitelübersicht

---

- 1 Einführung
- 2 Objektorientierung
- 3 Vertiefung C++**
- 4 Die Standard Template Library (STL)
- 5 Clean Code
- 6 Test-driven Development

# Kapitel 3: Vertiefung C++

---



31. Weitere C++ Keywords

32. Namespaces

33. Objektorientiertes Design

34. Übungsprojekt

# const

---

- Zur Definition von Variablen die nicht mehr verändert werden sollen (z.B. Konstanten)
- Keyword gibt an, dass die jeweilige Variable zur Laufzeit nicht mehr geändert werden kann

# const

---

- OOP: Methoden können `const` sein → Sie dürfen keine Member ändern
- OOP: Membervariablen (Argumente) können `const` sein → Diese dürfen von der Funktion/Methode nicht geändert werden
- `const`-Membervariablen müssen bei Objekterzeugung initialisiert werden (Initialisierungsliste des Konstruktors oder In-Class-Initialisierer im Header)

# constexpr

---

- Compile-Zeit-Auswertung → Wert/Berechnung wird bereits beim Kompilieren festgelegt
- Performance-Vorteil → Keine Berechnung zur Laufzeit nötig
- Implizit const → Wert ist unveränderlich

# Wann sollten Sie `const` / `constexpr` verwenden?

---

**Wann immer möglich!**

# Aufgabe: const

---

- Verwenden Sie das die Aufgabe constEdit  
(3\_Basics\_und\_Vertiefungen\_Cpp\31\_Vertiefung\311\_Keywords\tasks\constEdit\constEdit.cpp)
- Setzen Sie alles `const` was `const` gesetzt werden kann (und auch sollte).
- Versuchen Sie nicht zu raten sondern zu wissen.

# static

---

- Alle bisher genutzten Methoden/Variablen existierten auf Objektebene
- Zum Aufruf der Methoden muss vorher ein Objekt erzeugt sein
- Die Werte der Variablen können in jedem Objekt dieser Klasse unterschiedlich sein
- Mithilfe von `static` lassen sich **Variablen/Methoden definieren, die objektlos aufgerufen werden können**
- Dieser Wert/Methode ist dann in allen Objekten dieser Klasse gleich
- Die Methode hat nie Abhängigkeiten zu den Daten eines spezifischen Objekts

# static

---

- Alle bisher genutzten Methoden/Variablen existierten auf Objektebene
- Zum Aufruf der Methoden muss vorher ein Objekt erzeugt sein
- Die Werte der Variablen können in jedem Objekt dieser Klasse unterschiedlich sein
- Mithilfe von `static` lassen sich **Variablen/Methoden definieren, die objektlos aufgerufen werden können**
- Dieser Wert/Methode ist dann in allen Objekten dieser Klasse gleich
- Die Methode hat nie Abhängigkeiten zu den Daten eines spezifischen Objekts

# Aufgabe: static

---

Verwenden Sie die Aufgabenbeschreibung aus dem Git Repo:

**3\_Basics\_und\_Vertiefungen\_Cpp\31\_Vertiefung\311\_Keywords\tasks\task\_user\_static\_id\TASK\_USER\_STATIC\_ID.md**

# auto

- Kann anstelle des konkreten Typs bei einer Zuweisung angegeben werden, wenn direkt ein Wert zugewiesen wird
- Compiler ermittelt selbst den Typ
- Typsicherheit bleibt erhalten, d.h. Compiler warnt weiter wenn Typen nicht zusammen passen
- C++ Standard: AAA: Almost Always Auto
- Aber: Gefahr Lesbarkeit zu reduzieren!
- Besser: Auto nur wenn Typ auf rechten Seite klar definiert
- Auto nur wenn Lesbarkeit erhöht wird, z.B. bei zu langen Klassennamen

Welche Verwendung von auto ist sinnvoll?

```
auto numberOfUsers{5};  
int measValue{5.5};  
...  
auto humCalculator{HumidityCalculator(20, 50)};  
auto humCalculator = HumidityCalculator(20, 50);  
...  
auto measValue02{measValue}
```

# Friend class

- Variablen, die in der Regel `private` sein sollen, werden **von einer einzelnen spezifischen Klassen** benötigt
- hierzu wäre es nicht sinnvoll diese komplett als `public` zu definieren
- Die andere Klasse kann als `friend` deklariert werden
- `friend` Beziehungen gelten nur in eine Richtung
- Die Klasse `FriendClass` bekommt Zugriff auf die `private` Variablen der Klasse `Class`
- **Z.B. in Tests sinnvoll**

```
class Class
{
public:
    Class()
    : m_privateMember(5)
    {};

    ~Class()
    {};

    friend class FriendClass; //define FriendClass to be friend

private:
    int m_privateMember; //private member variable
};

class FriendClass
{
public:
    void testAccess(const Class& testObj)
    {
        std::cout << testObj.m_privateMember << std::endl; //has access to private member of Class
    }
};
```

# Aufgabe: keywords

---

Verwenden Sie die Aufgabenbeschreibungen aus dem Git Repo:

- **3\_Basics\_und\_Vertiefungen\_Cpp\31\_Vertiefung\311\_Keywords\tasks\BankAccounts\TASK\_BANKACCOUNTS.md**

# Kapitel 3: Vertiefung C++

---

31. Weitere C++ Keywords



32. Namespaces

33. Objektorientiertes Design

34. Übungsprojekt

# Namespaces - Motivation

---

- Annahme:  
Wir implementieren ein Algebra Programm und wollen darin eine Klasse `Vector` implementieren
- Problem: `Vector` gibt es schon! → Compiler Fehler
- Lösung: Namespaces! `Vector` der C++ Standard Template Library (STL) liegt im Namespace `std` und wird aufgerufen mit `std::vector` → Nur `std::vector` ist belegt, wir können unsere Klasse `Vector` nennen.
- Für große Projekte und oft wieder verwendeten Code unverzichtbar

Vorteile von Namespaces:  
Verhindern von Namenskollisionen  
Verbesserte Lesbarkeit (z.B. `algebra::vector`)

# Namespaces - Beispiel

- Bei großen Projekten: ALLES in Namespaces mit Namenskonventionen für die Namespaces, z.B. jedes Projekt einen eigenen Namespace
- Meine Empfehlung: In Namespaces nicht einrücken, da sonst oft ganze Files einfach ein oder zweimal eingerückt werden  
→ weniger Platz zum Vergleichen
- Aber Ende des Namespaces kommentieren, wofür die geschlossene Klammer steht

```
1  #include <vector>
2
3
4  namespace Algebra
5  {
6
7      class Vector
8      {
9
10     };
11
12 } // end namespace algebra
13
14
15 int main()
16 {
17     // create STL vector
18     std::vector<int> vec;
19     // create algebra vector
20     Algebra::Vector vec;
21
22     return 0;
23 }
```

Erster Buchstabe groß. Nach includes, vor class

Im Namespace nicht eingerückt

Ende kommentiert

# Namespaces – .hpp and .cpp files

```
3_Vertiefung_Cpp > namespaces > C++ main.cpp > main()
1  #include <vector>
2
3  #include "classInNamespace.hpp"
4
5
6  int main()
7  {
8
9      MyNamespace::ClassInNamespace myObject;
10
11      using MyNamespace::ClassInNamespace;
12
13      ClassInNamespace myObject;
14
15      return 0;
16  }
17
18

3_Vertiefung_Cpp > namespaces > h++ classInNamespace.hpp > ...
1
2  namespace MyNamespace
3  {
4
5      class ClassInNamespace
6      {
7
8      };
9
10 } // end namespace MyNamespace

3_Vertiefung_Cpp > namespaces > C++ classInNamespace.cpp > {} MyNamespace
1  #include "classInNamespace.hpp"
2
3  namespace MyNamespace
4  {
5
6      ClassInNamespace::ClassInNamespace()
7      {
8          // Implementation of Default Constructor
9      }
10
11 } // end namespace MyNamespace
```

- Namespaces können sich über mehrere Dateien erstrecken
- Wenn zusätzliche Klassen und Funktionen in einen Namespace eingefügt werden sollen oder die Implementierungen in der .cpp Datei, dann geht dies einfach über das erneute definieren des Namespaces.

# using namespace

---

- Ganze Namespaces können über `using` in den aktuellen Namespace eingebunden werden (z.B. `using namespace std;`)

**using namespace: Nur mit viel Vorsicht!**  
**Und niemals in Headern!**

- Auch Vorsicht vor `using namespace std;`
  - STL wird häufig genutzt: erscheint sinnvoll
  - Aber STL ist im stetigen Wandel
    - Namenskollisionen können entstehen obwohl eigener Code nicht geändert wurde
- Alternative: einzelne oft genutzte Ausdrücke aus `namespace` befreien durch z.B. `using std::cout;` (geht auch mit eigenen langen Namespaces)